

Deploying Mobile Nodes to Connect Wireless Sensor Networks Using Novel Algorithms

Chang Wu Yu, Elvis Chen, and Chun-Cheng Fang
Department of Computer Science and Information Engineering
Chung Hua University
Taiwan, R.O.C.
cwyu@chu.edu.tw

Abstract

Wireless sensor networks receive lots of attention due to its promising techniques and wide-ranging applications in recent years. The kind of network occasionally becomes disconnected due to initial uneven deployments or unpredictable failures or run out of battery of sensor nodes. However, sensor nodes with mobility then can be used in an addition deployment to reconnect the disconnected sensor networks. Theoretically, the augmenting geometric graph problem is defined here to model this kind of connectivity issues. The work proposes two novel algorithms: the graph-oriented algorithm and the divide-and-conquer algorithm to connect disconnected networks by using as less as possible mobile nodes. The first algorithm highly exploits traditional graph and geometry techniques including Fermat point, convex hull, nearest neighbor, minimum cost spanning tree, and graph contraction. Adopting a quite different approach, the second algorithm resolves the problem by dividing the deployed area and merging sub-solutions recursively. With respect to complexity issue, the graph-oriented algorithm takes $O(n^3)$ time; on the other hand, the divide-and-conquer algorithm requires $O(n \log n)$ time, where n is the size of vertex set of the given graph $G=(V, E)$. These proposed two algorithms have low time complexity and can be implemented in a centralized sensor network. Simulation results indicate that, when node density is sparse, the graph-oriented algorithm requires fewer nodes than the divide-and-conquer algorithm. On the contrary, when the node density is dense, the divide-and-conquer algorithm outperforms the graph-oriented algorithm. Both our proposed algorithms can reduce the number of required mobile sensor nodes dramatically.

Keywords: Mobile sensor networks, network connectivity, minimum cost spanning tree

1 Introduction

Wireless sensor networks receive lots of attention due to its promising techniques and wide-ranging applications in recent years. This kind of network consists of large number of low-cost, low-power, small-size, and multifunction sensor nodes which can sense and process data and communicate with other nodes in short distance [16]. Wireless sensor networks can be used for many application areas including health care [1], military [2], monitoring [2], etc. For example, on health care, we can place sensor nodes on devices or clothes to monitor the health state of patients or elders or children. On military, we can deploy sensor nodes on the battle field to sense the motivation of enemies. On monitoring, we also can deploy sensor nodes in a building for security issues.

Wireless sensor networks occasionally become disconnected due to initial uneven or random deployments or unpredictable failures or run out of battery of sensor nodes:

(1) Random deployment: The position of sensor networks need not be pre-determined. That indicates the initial topology of sensor networks might be disconnected. Consequently, some nodes can not communicate with some other nodes if without further adjustment.

(2) The energy of battery of some sensor nodes runs out: Since the energy of sensor nodes is limited, the lifetimes of sensor nodes may be quite different due to varied energy consumption rates. When some critical nodes are out of energy and then become malfunctioned, the network would be disconnected.

(3) Unpredictable failures of sensor nodes: Some sensor node may fail or be blocked due to physical damage or environmental interference. Such unpredictable failures may change network connectivity.

Figure 1 shows an example of a disconnected network consists of six disjoint connected sub-networks.

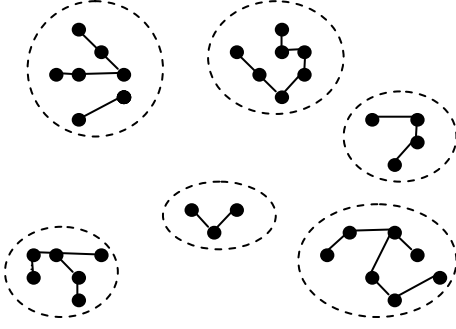


Figure 1. Six disconnected sub-networks.

Additional deployment of mobile sensor nodes then can reconnect the disconnected sensor networks, which become a critical issue in wireless sensor networks. Theoretically, the above problem can be modeled as a new graph problem described as follows. A geometric graph $G=(V, r)$ consists of nodes placed in 2-dimension space R^2 and edge set $E=\{(i, j) \mid d(i, j) \leq r, \text{ where } i, j \in V \text{ and } d(i, j) \text{ denotes the Euclidian distance between node } i \text{ and node } j\}$. Given a disconnected geometric graph, the augmenting geometric graph problem (AGGP) is adding the minimum number of nodes V such that the resulting graph $G=(V, r)$ is connected. Evidently, a mobile node with r communication radius can be represented by a node in a geometric graph (Figure 2).

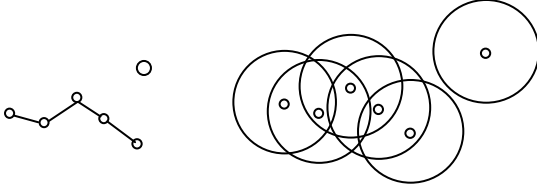


Figure 2. A geometric graph G with its representing network.

The connectivity number of G , written $k(G)$, is the minimum size of a vertex set S such that $G-S$ is disconnected or has only one vertex. A graph G is k -connected if its connectivity is at least k . In other words, the AGGP is aimed at increasing the connectivity number from 0 to 1 by adding the minimum number of new nodes. The AGGP has been formulated to a generalized Steiner minimum tree problem and shown to be NP-hard [24]. Thus, some approximation and heuristic algorithms have been designed for AGGP [21-24].

In this paper, we propose two novel algorithms: the graph-oriented algorithm and the divide-and-conquer algorithm to connect disconnected networks by using as less as possible mobile nodes. The first algorithm highly exploits traditional graph and geometry techniques including Fermat point, convex hull, nearest

neighbor, minimum cost spanning tree, and graph contraction. Adopting a quite different approach, the second algorithm resolves the problem by dividing the deployed area and merging sub-solutions recursively. The graph-oriented algorithm takes $O(n^3)$ time; on the other hand, the divide-and-conquer algorithm requires $O(n \log n)$ time, where n is the size of vertex set of the given graph $G=(V, E)$. These proposed two algorithms have low time complexity and can be implemented in a centralized sensor network. Simulation results indicate that, when node density is sparse, the graph-oriented algorithm requires fewer nodes than the divide-and-conquer algorithm. On the contrary, when the node density is dense, the divide-and-conquer algorithm outperforms the graph-oriented algorithm. Both our proposed algorithms can reduce the number of required mobile sensor nodes dramatically.

The rest of the paper is organized as follows. The next section presents related work, followed by some definitions and preliminaries used in Section 3. The two algorithms proposed are described in detail in Section 4. Next, Section 5 presents the simulation results. Finally, Section 6 concludes this paper.

2 Related work

Lin and Xue [24] have shown the AGGP is NP-hard and gave a 5-approximation algorithm. In [23], Chen *et al.* showed that Lin and Xue's algorithm has performance ratio exactly 4, and they also proposed a $O(n^4)$ -time approximation algorithm with ratio at most 3. In [20], Cheng *et al.* proposed a $O(n^3)$ -time approximation with performance ratio at most 3, and a randomized approximation with performance ratio at most 5/2. Recently, Chang and Jan have proposed a $O(n^2)$ -time heuristic algorithm for the AGGP [21].

The AGGP is quite similar to the *group Steiner tree problem*: Given a undirected graph $G=(V, E)$, a nonempty subset $N=(N_1, N_2, N_3, \dots, N_k)$ and $N \subseteq V$, find a tree $T_G(N)$ spanning at least one vertex from N_i , for $i=1$ to k , while minimizing the sum of the corresponding edge costs. This problem had been proved as a NP-hard problem [[14]].

Rengarajan *et al.* [[4]] considered how to connect sub-networks with power control. They try to change the communication radius of sensor nodes to connect with other nodes. Since the energy of sensor nodes is limited, the approach will shorten the lifetime of network significantly.

In [5], Hellbrück and Fischer presented the MINE concept to maintain the connectivity of sensor network. They had proposed a protocol that uses mobile nodes to maintain the network connectivity. However, they do not mention how

to reduce the number of needed mobile nodes that is used to connect the disconnect networks.

3 Preliminary

In a planar space, a point is called the *Fermat point* if it minimizes the sum of the distances from the point to every vertex of a triangle [19]. The Fermat point can be constructed (on the outside of the triangle) by drawing two equilateral triangles on two side of the triangle and connecting the opposite vertices of the original triangle and the created equilateral vertices, where an *equilateral triangle* is a triangle with all three sides of equal length. Note that the two diagonals intersect in the Fermat point (Figure 3). If there is an interior angle of this triangle is greater than 120 degree, the Fermat point is exactly the vertex with the angle ≥ 120 degree (Figure 4).

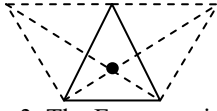


Figure 3. The Fermat point of a triangle.



Figure 4. The Fermat point of a triangle with an interior angle greater than 120 degree.

A simple polygon is *convex* if, given any two points in the polygon (in boundary or interior), all points on the line segment between them are contained in the polygon. The *convex hull* of a set of points is the smallest convex polygon containing them (Figure 5).

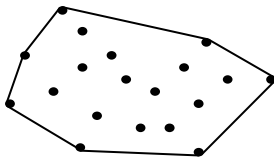


Figure 5. A convex hull

A *graph* $G=(V, E)$ consists of a finite nonempty vertex set V and edge set E of unordered pairs of distinct vertices of V . A *connected graph* means that any vertex of the graph has at least one path to any other vertex. Otherwise, it is *disconnected*. A *connected component* means that the biggest connected sub-graph of a graph. A *tree* is a connected graph without close loops (or cycles). A *minimum cost spanning tree* is a tree that connects all nodes in a connected and weighted graph with the minimum cost. Figure 6 depicts an example.

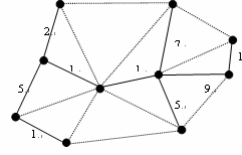


Figure 6. A minimum cost spanning tree.

An edge e of G is said to be *contracted* if it is deleted and its ends are identified. A subgraph of G is said to be *contracted* if all its edges are contracted successively in any order. Some graph definitions and notations given here can be found in [15].

4 Novel deploying algorithms

In this work, we assume that each node equipped with a GPS device and always know its own position information. All nodes in the same connected subnetwork also can find each other by elementary broadcasting method. We also assume that there exists at least one node, named network head, in each connected subnetwork so that the node can transmit the position information of every node in the connected subnetwork to a designated process server. After computing and analyzing the position information of the whole network, the server will command mobile nodes to move to connect all disconnected sub-networks. In the following subsections, we will propose two algorithms including the graph-oriented algorithm and the divide-and-conquer algorithm for deploying mobile sensor nodes so that the resulting network is connected.

4.1 The graph-oriented algorithm

The first algorithm highly exploit traditional graph and geometry techniques including Fermat point, convex hull, nearest neighbor, minimum cost spanning tree, and graph contraction. Remind that a sensor network can be viewed as a geometry graph. Given a disconnected sensor network N with its corresponding geometry graph $G=(V, E)$, the first step is finding out the connected components of G . By the definition of geometry graph, we then can represent each mobile node by using a point in a plane. Next, the algorithm finds out the point sets each of which represents the corresponding nodes of each sub-network, and then determines the corresponding convex hull of each point set (as shown in Figure 7).

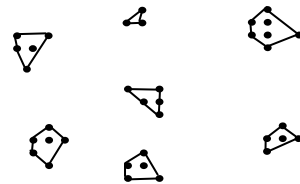


Figure 7. Finding convex hulls of connected components.

The obtained convex hull is the small size of convex polygon which contains every points of the subnetwork. Roughly, each subnetwork can be viewed as a polygon (i.e. the constructed convex hull). Note that every once in a while two polygons may intersection each other or one is contained in another one. Since the nodes on the convex hull is closer to other subnetworks than that in the convex hull (Figure 7), we can focus on nodes on convex hull but ignore that in a convex hull in determining where additional nodes should be deployed to connect the whole network.

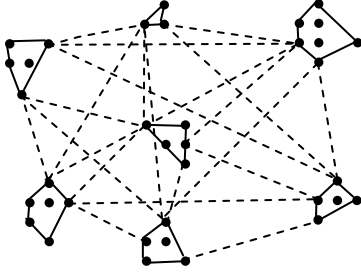


Figure 8. The supergraph G .

The remaining effort is devoted to connecting all these polygons (representing the subnetworks) by using the least number of mobile nodes. Thus, the next step tries to find such connections between every two different subnetworks. The most straightforward way is finding a shortest straight line between two polygons in the plane initially although there might exists no such direct communication; latter, our protocol will replace such straight line by lining a couple of nodes up. For example, given a straight line with length l between two nodes with communication radius r , we may deploy $\lceil l/r \rceil$ nodes between them to form a communication path. We can represent this kind of distances between polygons by using a complete graph, called *supergraph* $G=(V, E)$, such that each polygon is represented as a vertex v in V and the shortest distance between two polygon's vertices as the weight of the corresponding edge e in E , even when two convex hulls contain or intersect each other. An example is shown in Figure 9. The protocol connects these polygons by finding a minimum cost spanning tree T on the above weighted supergraph G (Figure 9).

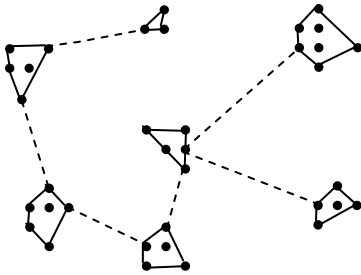


Figure 9. The minimum cost spanning tree of Figure 6.

The cost (i.e. the number of required mobile nodes) may be reduced furthermore by using Fermat point technique on T . First we find any internal node t_1 in T with the minimum degree (e.g. the minimum degree of internal nodes in T is 2 in Figure 10).

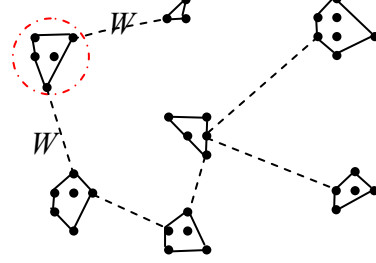


Figure 10. Selecting an internal node with the minimum degree.

When the degree of the selected node is two, our protocol constructs a triangle Δabc with the minimum sum of three sides where a , b , and c are nodes selected from three different convex hulls containing nodes a , b , and c , respectively. Next, the Fermat point F of the triangle Δabc is computed as shown in Figure 11.

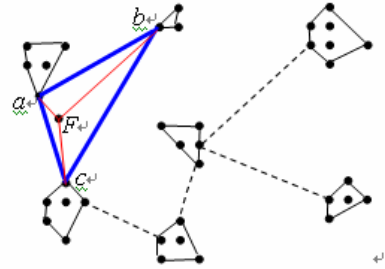


Figure 11. When the degree of the selected node is two, our protocol constructs a triangle Δabc with the minimum sum of three sides.

The number of nodes to fulfill the original two lines is $\lceil w_1/r \rceil + \lceil w_2/r \rceil$, where w_1 and w_2 are the shortest distance between the selected convex hull and its two neighboring convex hulls; on the other hand, the number of nodes for connecting the Fermat point by using three lines is $\lceil |aF|/r \rceil + \lceil |bF|/r \rceil + \lceil |cF|/r \rceil$. The protocol will always select the least number of nodes to connect the corresponding three subnetworks. Specifically, if $\lceil w_1/r \rceil + \lceil w_2/r \rceil \leq \lceil |aF|/r \rceil + \lceil |bF|/r \rceil + \lceil |cF|/r \rceil$, we connect the original two lines. Otherwise, we connect a , b , and c to F at the same time. These three subnetworks then merge into a larger network, and the protocol also construct the convex hull of the new network to replace their original convex hulls as shown in Figure 12.

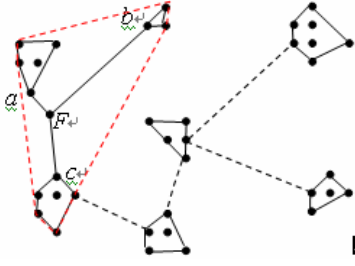


Figure 12. Three subnetworks merge into a larger network and a new convex hull is created.

Finally, these three nodes in original tree structure T are contracted into a new node to generate another new tree. The graph-oriented algorithm connects all subnetworks by iterating the above steps similarly until the resulting tree is shrunk into a single node.

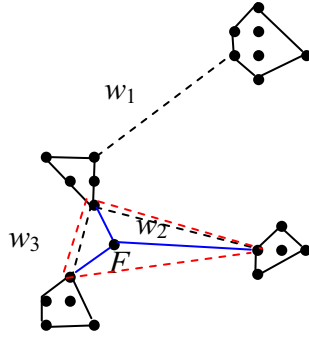


Figure 13. The FERMAT point of three sub-networks.

Suppose that the degree k of the selected node in T is greater than or equal to three and their associated weights are $\{w_1, w_2, \dots, w_k=w_0\}$ which are arranged clockwise, the graph-oriented algorithm will select two consecutive edges with the minimum sum of weights (i.e. $\min \{w_i+w_{i+1}\}$ for $i=1$ to k .) for constructing a FERMAT point and further shrinking process (Figure 13). The basic idea behind the algorithm step is merging small and near subnetworks first.

The algorithmic form of the graph-oriented algorithm is described as follows:

The graph-oriented algorithm.

Input: A disconnected sensor network N with its corresponding geometry graph $G=(V, E)$.

Output: A set of additional nodes such that the resulting sensor network is connected.

Step 1: Determine whether the given network is connected or not.

Step 2: If the given network is not connect, then identify all maximal connected subnetwork and find its convex hull and remove all nodes in its convex hull, for every connected sub-network.

Step 3: For every two different convex

hulls, find a single vertex in each convex hull such that the shortest distance between them is minimized. Construct a *supergraph* $G^*=(V^*, E^*)$, such that each convex hull (polygon) is represented as a vertex v in V^* and the shortest distance between two polygon's vertices as the weight of the corresponding edge e in E^* .

Step 4: Find a minimum cost spanning tree T in G^* .

Step 5: When $|T| \geq 2$, find any internal node v of T with the minimum degree. Otherwise, the algorithm terminates.

Step 6: If $d(v)=k \geq 2$ and the neighboring nodes of v are $\{v_0, v_1, \dots, v_{k-1}\}$, which are arranged clockwise in the plane, then select an integer i such that $w_{i \bmod k} + w_{(i+1) \bmod k}$ is minimized where w_i is the weight of edge (v, v_i) in G^* , for $0 \leq i \leq k-1$.

Step 7: Constructs a triangle Δabc with the minimum sum of three sides where a, b , and c are nodes selected from three different convex hulls represented by $v, v_{i \bmod k}$, and $v_{(i+1) \bmod k}$.

Step 8: Find the FERMAT point F of Δabc .

Step 9: If $\lceil w_1/r \rceil + \lceil w_2/r \rceil \leq \lceil aF/r \rceil + \lceil bF/r \rceil + \lceil cF/r \rceil$ then deploy $\lceil w_1/r \rceil + \lceil w_2/r \rceil$ nodes to connect the corresponding subnetwork of v to that of $v_{i \bmod k}$, and the corresponding subnetwork of v to that of $v_{(i+1) \bmod k}$. Otherwise, deploy $\lceil aF/r \rceil + \lceil bF/r \rceil + \lceil cF/r \rceil$ nodes to connect a to F and b to F and c to F respectively.

Step 10: Contract the subgraph formed by $v, v_{i \bmod k}$, and $v_{(i+1) \bmod k}$ in T and construct the corresponding convex hull of the new connected subnetwork.

Step 11: Go to Step 5.

Evidently, the above algorithm connects all disconnected subnetworks by deploying additional nodes. The execution time of the algorithm is discussed in the next theorem.

Theorem 1: The graph-oriented algorithm takes $O(n^3)$ time, where n is the size of vertex set of the given graph $G=(V, E)$.

Proof: The graph-oriented algorithm can be implemented as follows. Step 1 can be implemented in $O(n+m)$ time by finding connected component by using depth-first search or breadth first search. Since convex hulls can be found in $O(n \log n)$ time [17], Step 2 takes $O(n \log n)$ time. Step 3 can be done in $\Theta(n^3)$ time since the minimum distance between two nonintersecting convex polygons can be found in $\Theta(n)$ time [18] and there are at most $O(n^2)$ pairs of convex hulls. Step 4 can be implemented in $O(m \log m)$ time by applying Kruskal' algorithm [12], where m is the size of the edge set. Step 5 can be implemented in $O(m)$ time by computing the degree for each node and finding the minimum degree among them. Step 6 can be implemented in $O(k \log k)$ time by first sorting the incident edges of the node selected

in Step 5 and selecting an integer i such that $w_{i \bmod k} + w_{i+1 \bmod k}$ is minimized, where k is the degree of the node. Step 7 needs $O(\log^3 n)$ time by binary searching the desired triangle among the corresponding three convex hulls. Evidently, we can execute both Step 8 and Step 9 in constant time. Similarly, Step 10 can be implemented in $O(n \log n)$ time. Since Step 5-10 form a loop, we require totally $O(n \log n)$ time to execute the loop for one time. Each time the size of the tree can be reduced by two; therefore, we take $O(n \log n) \times O(n) = O(n^2 \log n)$ time to finish the loop. To sum up, the graph-oriented algorithm takes $O(n^3)$ time.

4.2 The Divided-and-Conquer Algorithm

The second algorithm exploits the divide-and-conquer paradigm. Initially, the deployed area is organized in a grid pattern consisting of $2r \times 2r$ unit squares, where r is the transmission radius of sensor nodes. For simplicity, we assume there are $2^k \times 2^k$ squares arranged in 2^k rows and 2^k columns.

The skeleton of the divide-and-conquer algorithm is described as follows:

Input: A disconnected sensor network N with its corresponding geometry graph $G=(V, E)$.

Output: A set of additional nodes such that the resulting sensor network is connected.

The dividing step: Divides the deployed area into quarters (as shown in Figure 14).

The conquering step: If each quarter is bigger than a $2r \times 2r$ square, call the divide-and-conquer algorithm recursively. When the quarter is a $2r \times 2r$ square, determine whether the whole network is connected or not. If the whole network is not connected, deploy the minimum number of additional nodes to connect the whole network.

The merging step: Merge the four connected subnetworks in four quarters into a larger connected network by deploying additional number of nodes.

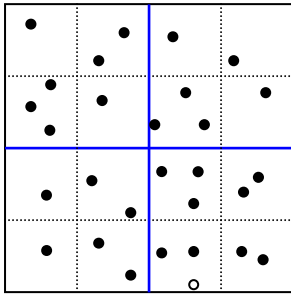


Figure 14. The divide-and-conquer algorithm divides the deployed area into four quarters recursively.

More detailed descriptions of Conquering Step

and Merging Step are described as follows. Conquering Step deploys l nodes in a $2r \times 2r$ square to connect disconnected subnetworks. Note that we will conclude that $1 \leq l \leq 4$ according to the following four cases and discussions.

Case 1: $l=1$. If there is a node in the square such that the shortest distance from each subnetwork to the center of the square is shorter than r , then we can deploy only one node at the center subnetworks (as shown in Figure 15(a)) to reach our goal.

Case 2: $l=2$. If there are two nodes whose distance is greater than $2r$ and located in different subnetworks, then they must be located in opposite corners of the square. In this case, we can deploy two new nodes along the diagonal of the square as shown in Figure 15(b) or near the boundaries of that as shown in Figure 15(c).

Case 3: $l=3$. If there are four nodes belonging to different subnetworks and located in the four corners of the square, we can line up three new nodes as shown in Figure 15(d).

Case 4: $l=4$. In worst case, we deploy at most four nodes as shown in Figure 13(e) to connect all other nodes in the square

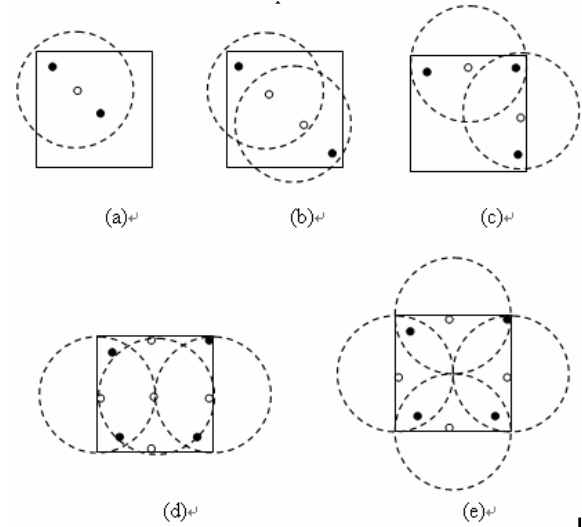


Figure 15. There are four ways to connect nodes in a square.

The last step, Merging Step, merges four obtained connected subnetworks into one by deploying new nodes. From each of four subnetworks, our algorithm first selects three extreme nodes which are closest to the center of the bigger square as shown in Figure 16. We require finding out three straight lines with minimum sum of lengths to connect these four subnetworks as depicted in Figure 17.

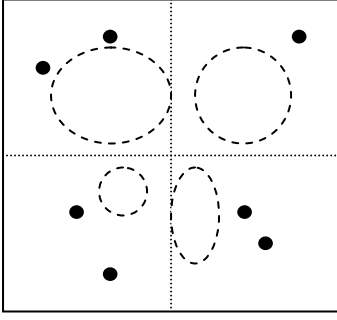


Figure 16. Selecting three extreme nodes which are closest to the center of bigger square.

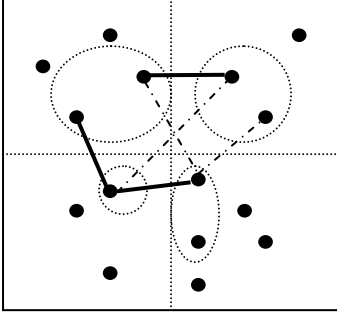


Figure 17. Finding out three straight lines with minimum sum of lengths to connect the whole network.

Theorem 2: The divide-and-conquer algorithm requires $O(n \log n)$ time.

Proof: The dividing step trivially takes constant time. The merge step of the divide-and-conquer algorithm needs $O(n)$ time to select the possible three extreme nodes; therefore, the overall time required the divide-and-conquer algorithm for n nodes is $f(n)=4(f(n/4))+O(n)$. After solving the recurrence, we can obtain that the divide-and-conquer algorithm requires $O(n \log n)$ time.

5 Simulation results

To investigate the performance of our proposed algorithms including the graph-oriented algorithm and the divide-and-conquer algorithm, we conduct simulations to compute the number of required mobile nodes with the places where these nodes should be deployed.

The simulation environment is specified by a 500 by 500 square and 500 by 250 rectangular regions with a range of numbers of deployed nodes from 10 to 120 with two different communication radiuses: 40 and 56. Each of these deployed nodes is placed randomly independent and identically distributed (i.i.d.) inside the given region. Each simulation scenario represents an average of 100 test sample runs.

The first scenario consists of a 500×500 square with communication range $r=40$. The simulation results are depicted in Figure 18. As anticipated, the graph-oriented algorithm outperforms the divide-and-conquer algorithm by deploying fewer nodes. In this scenario, in the beginning, both require slightly more nodes to connect the disconnected network when the number of deployed nodes increases. However, the required nodes of the graph-oriented algorithm reach its peak when the number of deployed nodes is about 80, but that of the divide-and-conquer algorithm is still rising.

Next, the second scenario considers a narrower area with same communication range and deployed nodes. Note that the gap between these two algorithms becomes smaller; and the number of additional nodes is slightly decreased as the number of deployed nodes increases (Figure 19). It seems reasonable because when deployed area becomes narrower then the empty space between neighboring disconnected subnetworks becomes small; as a result, the required additional nodes is decreased.

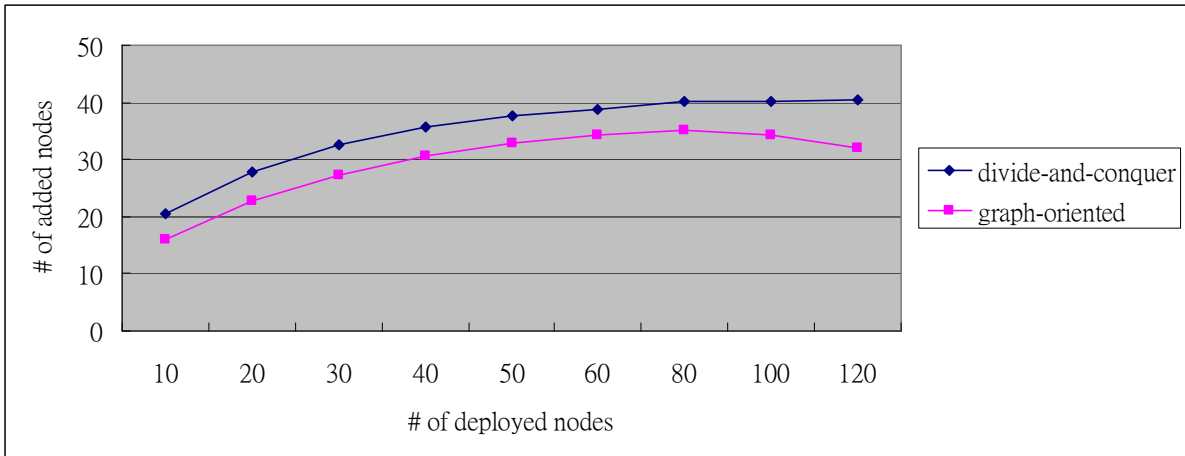


Figure 18. The added nodes when a 500×500 square with $r=40$

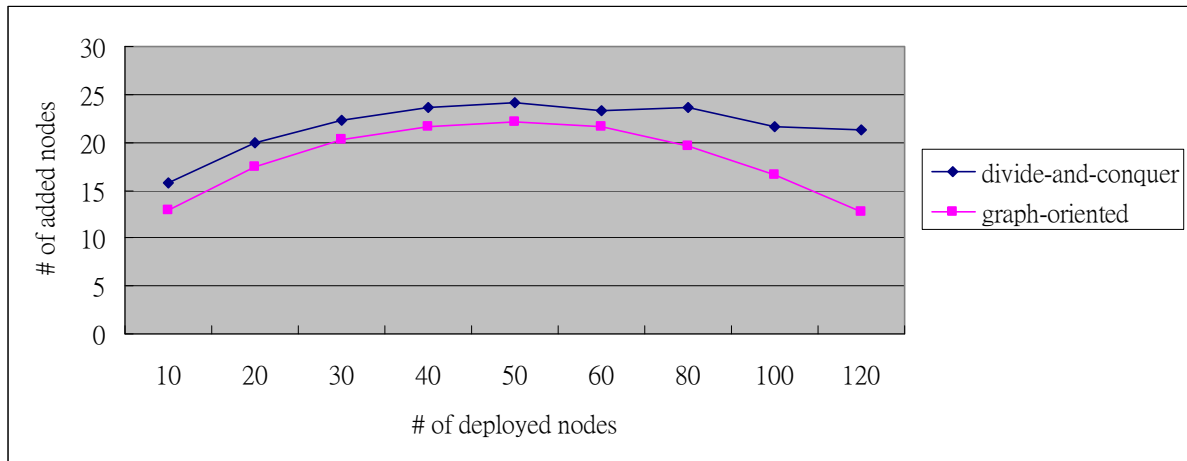


Figure 19. The added nodes when a 500×250 square with $r=40$.

The third (Figure 20) and forth scenario (Figure 21) consider a larger communication range with $r=56$. In these cases, the divide-and-conquer algorithm is matched against the graph-oriented algorithm due to higher node density of the given network. Similarly, the number of added nodes is smaller when a thinner area is considered.

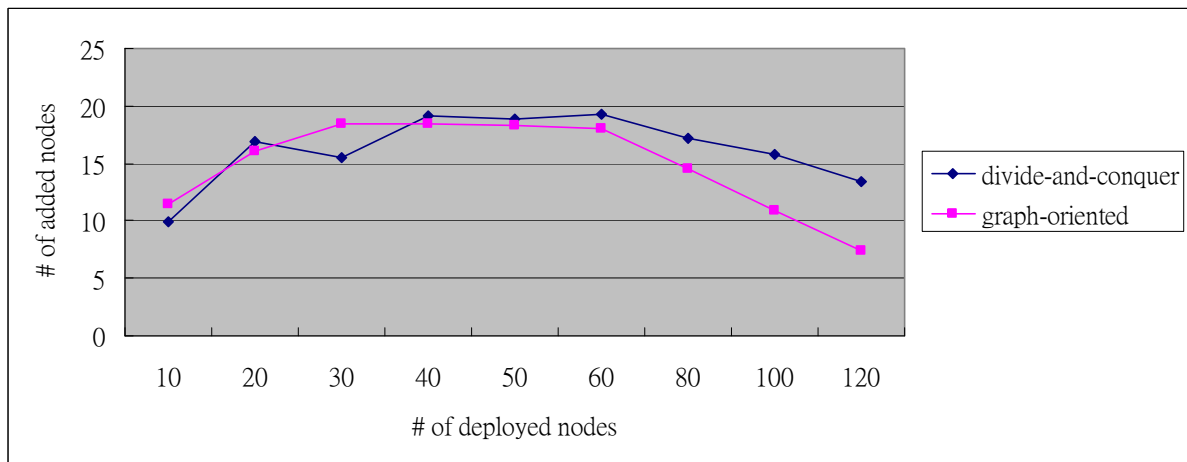


Figure 20. The added nodes when a 500×500 square with $r=56$.

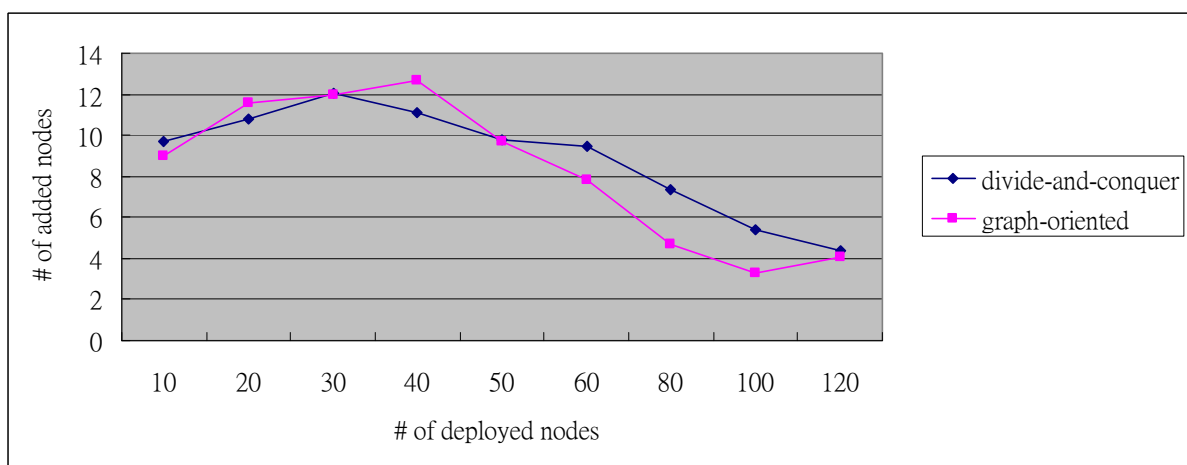


Figure 21. The added nodes when a 500×250 square with $r=56$.

According to the simulation results, we suggest to exploit the divide-and-conquer algorithm when the disconnected network is almost connected due to its lesser time complexity and good performance. However, when the node density of the given disconnected network is sparse, the graph-oriented algorithm deploys

fewer nodes than the divide-and-conquer algorithm does due to its sophisticated techniques

6 Conclusions

This work presents two algorithms to connect the disconnected wireless sensor networks by using additional mobile sensor networks as less as possible. The graph-oriented algorithm is more complicated than the divided-and-conquer algorithm but requires less mobile nodes especially when the node density is sparse. In future, it is interesting to design a more efficient algorithm than the graph-oriented algorithm deserves further consideration. Moreover, it is believed that the techniques exploited in this work can be applied to coverage problems of sensor networks.

Reference:

- [1] Haowei Bai, Mohammed Atiquzzaman, and David Lilja, "Wireless Sensor Network for Aircraft Health Monitoring," *Proceedings of the First International Conference on Broadband Networks (BROADNETS'04)*, 2004, pp. 748-750.
- [2] Chee-Yee Chong and SRIKANTA P. KUMAR "Sensor Networks: Evolution, Opportunities, and Challenges," *Proceedings of the IEEE*, vol. 91, no. 8, 2003.
- [3] Kay Römer, Friedman Mattern, and Eeth Zurich, "The design space of wireless sensor networks," *IEEE Wireless Communications*, vol. 11, pp. 54-61, December 2004.
- [4] Balaji Rengarajan, Jeremy K. Chen, Sanjay Shakkottai, and Theodore S. Rappaport, "Connectivity of Sensor Networks with Power Control," *Signals, Systems and Computers*, vol. 2, pp. 1691-1693, 2003.
- [5] Horst Hellbrück and Stefan Fischer, "MINE and MILE: Improving Connectivity in Mobile Ad-Hoc Networks," *ACM SIGMOBILE Mobile Computing and Communications Review*, vol. 8, pp. 19-36, 2004.
- [6] X. Wang, G. Xing, Y. Zhang, C. Lu, R. Pless, and C. D. Gill. "Integrated coverage and connectivity configuration in wireless sensor networks" *The First ACM Conference on Embedded Networked Sensor Systems*, 2003, pp. 28-39.
- [7] Ghosh. "Estimating Coverage Holes and Enhancing Coverage in Mixed Sensor Networks" *Proceedings of the 29th Annual IEEE International Conference on Local Computer Networks (LCN)*, 2004, pp. 742-1303.
- [8] S. Zhou, M.-Y.Wu, and W. Shu, "Finding optimal placements for mobile sensors: wireless sensor network topology adjustment," *Proceedings of the IEEE Symposium on Circuits and Systems*, 2004, vol. 2, pp. 529 - 532.
- [9] Thomas H. Corman, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, The MIT Press, 1990.
- [10] Douglas B. West, *Introduction to Graph Theory*, Prentice Hall, 2001.
- [11] M. Smid, *Handbook of Computational Geometry*, Amsterdam, Netherlands: North-Holland, 2000.
- [12] Ellis Horowitz, Strtaj Sahni, and Susan Anderson-Freed, *Fundamentals of Data Structures in C*, W H Freeman & Co, 1995.
- [13] Ian F. Akyildiz, Weilian Su, Yogesh Sankarasubramaniam, and Erdal Cayirci, "A Survey on Sensor Networks," *IEEE Communications Magazine*, vol. 40, issue 8, pp. 102- 114, 2002.
- [14] Frank K. Hwang, Dana S. Richards, and Pawel Winter, *The Steiner Tree Problem*, N.H 1992.
- [15] J.A. Bondy and U.S.R. Murty, *Graph Theory with Applications*, Macmillan, London, 1976.
- [16] I.F. Akyildiz, W. Su, Y. Sankarasubramaniam, and E. Cayirci, "Wireless sensor networks: a survey," *Computer Networks*, vol. 38, pp. 393-422, 2002.
- [17] R. L. Graham, "An efficient algorithm for determining the convex hull of a finite planar set," *Information Processing Letters*, vol. 1, pp. 132-133, 1972.
- [18] G. T. Toussaint, "An optimal algorithm for computing the minimum vertex distance between two crossing convex polygons," *Computing*, vol. 32, pp. 357-364, 1984.
- [19] N. D. Kazarinoff, *Geometric Inequalities*. New York: Random House, 1961.
- [20] X. Cheng, D.-Z. Du, L. Wang, and B. Xu, "Relay sensor placement in wireless sensor networks," accepted for publication in *ACM WINET*.
- [21] J.-H. Chang and R.-H. Jan, "An efficient relay sensors placing algorithm for connectivity in wireless sensor networks," *Lecture Notes in Computer Science*, vol. 4096, pp. 874-883, 2006.
- [22] Mandoiu and Z. Zelikovsky, "A note on the MST heuristic for bounded edge-length Steiner trees with minimum number of

- Steiner Points,” *Information Processing Letters*, vol. 75, pp.165-167, 2000.
- [23] D. Chen, D.-Z. Du, X. Xu, G.-H. Lin, L. Wan, and G. Xue, “Approximations for Steiner trees with minimum number of Steiner points,” *Theoretical Computer Science*, vol. 262, pp. 83-99, 2001.
- [24] G. Lin and G. Xue, “Steiner tree problem with minimum number of Steiner points and bounded edge-length,” *Information Processing Letters*, vol. 69, pp. 53-57, 1999.